

LTI Systems with Random Inputs

Predict output statistics without predicting the exact waveform

$$\mu_y = \mu_x \sum h[k]$$

$$r_{yy} = r_{xx} * h * h_{\text{rev}}$$

Roadmap and learning objectives

75 minutes • mean → input/output correlation → white-noise coloring → augmentation

- Propagate the mean of a WSS process through an LTI system.
- Derive input-output and output autocorrelation relationships under a stated correlation convention.
- Compute output variance from the input covariance / autocorrelation.
- Explain why an LTI filter turns white noise into colored noise.
- Use controlled noise models for PyTorch data augmentation.

0–20 min

Mean and variance

20–50 min

Correlation dynamics

50–65 min

White noise → colored noise

65–75 min

PyTorch augmentation

01

Mean and variance through an LTI system

Linearity lets expectation pass through convolution.

Output mean is controlled by the DC gain

For a WSS input, $E\{X[n-k]\} = \mu_x$ for every k .

$$Y[n] = \sum_k h[k] X[n-k]$$

$$\mu_y = E\{Y[n]\} = \mu_x \sum_k h[k] = \mu_x H(e^{j0})$$

DC gain = 1

Mean passes through unchanged.

DC gain = 0

A constant mean is removed; e.g., a difference / highpass filter.

DC gain > 1

The mean is amplified by the same gain.

CONCEPT CHECK

Input $X[n]$ has mean 3. The filter is $h[n] = \{1, -1\}$. What is μ_Y ?

Good answer

$\mu_Y = 3(1-1) = 0$. The first-difference filter has zero DC gain, so it removes the constant mean component.

Variance needs second-order information

Mean alone is not enough to predict random fluctuations at the output.

$$\sigma_Y^2 = E\{|Y[n] - \mu_Y|^2\}$$

$$\sigma_Y^2 = \sum_k \sum_l h[k] h^*[l] c_{xx}[k-l]$$

- $c_{xx}[m]$ is the input autocovariance (mean removed).
- Correlated input samples can add constructively or destructively through the filter.
- For white noise, all off-zero covariance lags vanish, giving a much simpler result.

White-noise special case

$$\sigma_Y^2 = \sigma_x^2 \sum_k |h[k]|^2$$

02

Input-output correlation dynamics

Convolution of waveforms becomes convolution of correlation sequences.

Correlation convention: define it before using reversal formulas

Different texts/software place the lag on different arguments.

This lecture: $r_{xy}[m] = E\{X[n] Y^*[n+m]\}$

Alternate: $\tilde{r}_{xy}[m] = E\{X[n+m] Y^*[n]\}$

Then: $r_{xy}[m] = (r_{xx} * h^*)[m]$ for $y = h * x$

Then: $\tilde{r}_{xy}[m] = (r_{xx} * h_{\text{rev}}^*)[m]$

Why this matters

The user-facing formula $r_{xy} = r_{xx} * h[-m]$ is correct under the alternate lag convention, but not under the Lecture 15 definition $r_{xy}[m] = E\{X[n] Y[n+m]\}$. Keep one convention consistent throughout.

Notation

For real-valued filters, $h^* = h$. Define $h_{\text{rev}}[m] = h[-m]$.

Deriving $r_{xy}[m]$ with the Lecture 15 convention

Move expectation through the deterministic sum.

$$Y[n+m] = \sum_k h[k] X[n+m-k]$$

$$r_{xy}[m] = E\{X[n]Y^*[n+m]\}$$

$$= \sum_k h^*[k] r_{xx}[m-k]$$

$$= (r_{xx} * h^*)[m]$$

Interpretation

The system smears the input autocorrelation by the same impulse response that filters the signal.

Sanity check

If $h[n] = \delta[n]$, then $r_{xy}[m] = r_{xx}[m]$.

Complex signals

The conjugation follows from the correlation definition.

Output autocorrelation: filter once on each side

The output contains two filtered copies inside the second-order product.

$$r_{yy}[m] = r_{xx}[m] * h[m] * h_{\text{rev}}^*[m]$$

$$r_{hh}[m] = h[m] * h_{\text{rev}}^*[m]$$

$$\text{Therefore: } r_{yy}[m] = r_{xx}[m] * r_{hh}[m]$$

- For real $h[n]$, $h_{\text{rev}}^*[m] = h[-m]$.
- r_{hh} is deterministic autocorrelation of the impulse response.
- At $m = 0$, $r_{yy}[0]$ gives average output power; subtract $|\mu_y|^2$ for variance.

CONCEPT CHECK

If $h[n] = \delta[n] + 0.5\delta[n-1]$, does a white-noise input remain white?

Good answer

No. Adjacent output samples share one of the same input samples, so they become correlated. The filter “colors” the white noise.

A matrix view: filtering transforms covariance

Useful bridge to linear algebra and ML.

$$y = Hx$$

$$C_y = H C_x H^H$$

- If $C_x = \sigma^2 I$ (white), then $C_y = \sigma^2 H H^H$.
- Filtering creates off-diagonal covariance unless H is trivial / orthogonal in a special way.
- This is the same covariance propagation idea used in estimation, Kalman filtering, and uncertainty-aware ML.

Connection

The scalar correlation formulas are the Toeplitz / infinite-length version of $C_y = H C_x H^H$.

03

White noise and filtering

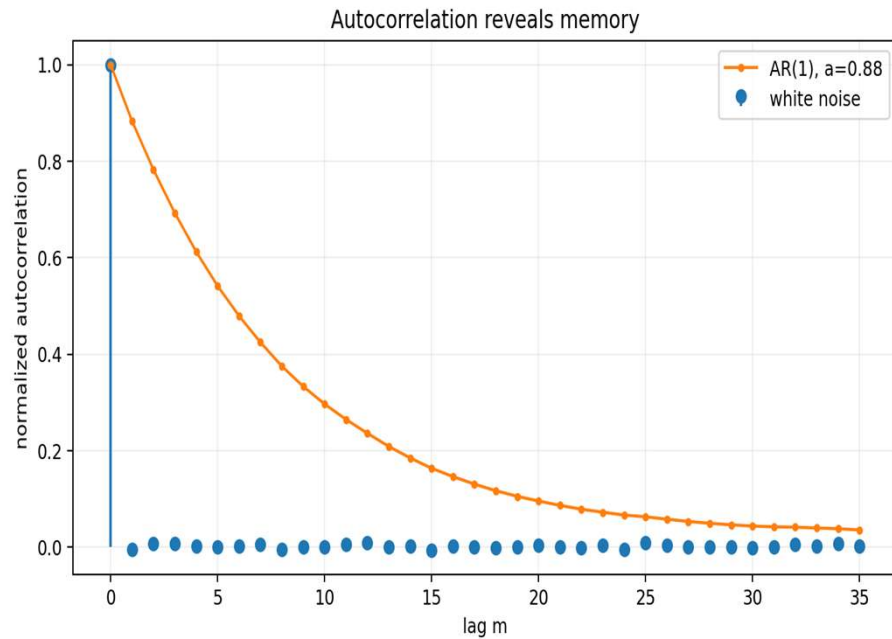
“White” means flat second-order spectrum / zero correlation away from lag 0.

White noise: uncorrelated across nonzero lags

Zero-mean white noise has one nonzero autocorrelation sample.

$$r_{xx}[m] = \sigma^2 \delta[m]$$

- $r_{xx}[0] = \sigma^2$: finite average power.
- $r_{xx}[m] = 0$ for $m \neq 0$: no linear second-order memory.
- “White” does not mean Gaussian. Gaussian white noise is one important special case.



Filtering white noise creates colored noise

The output autocorrelation becomes a scaled autocorrelation of $h[n]$.

$$r_{\gamma\gamma}[m] = \sigma^2 r_{hh}[m]$$

$$\sigma_\gamma^2 = r_{\gamma\gamma}[0] = \sigma^2 \sum_k |h[k]|^2$$

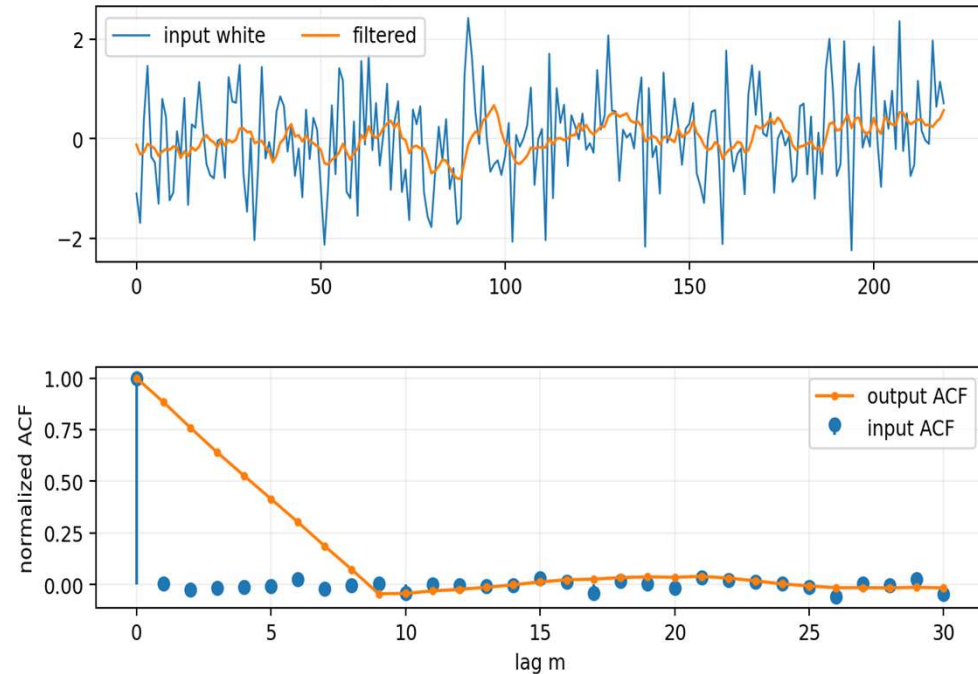
Meaning of “colored”

The filter introduces correlation across time, so some frequencies become stronger than others.

Example

A moving-average lowpass makes neighboring output samples similar and suppresses high-frequency noise.

FIR filtering turns white noise into correlated noise



Frequency-domain preview: the filter shapes the noise PSD

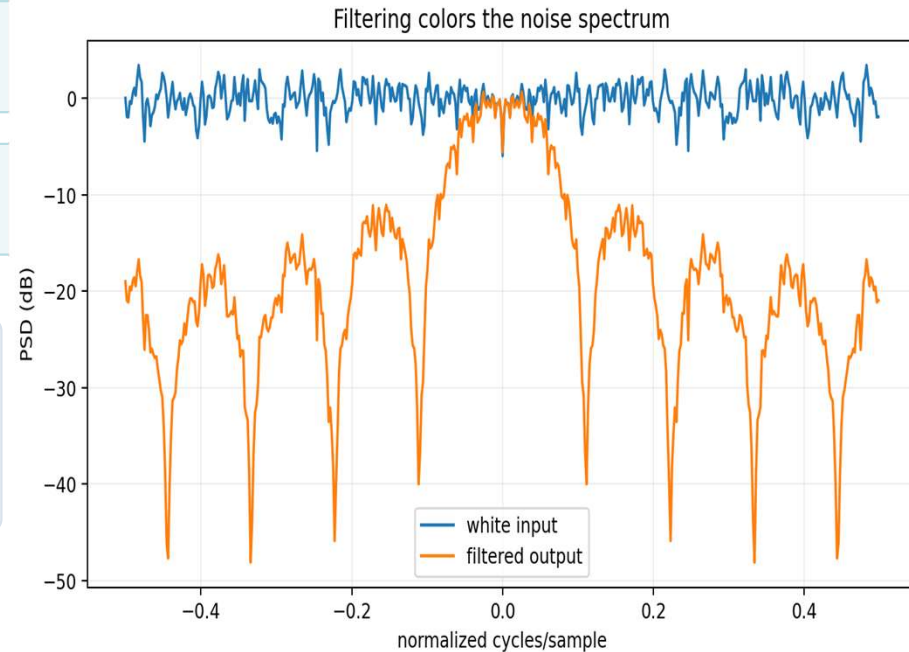
Second-order time-domain convolution becomes multiplication in frequency.

$$S_{yy}(e^{j\omega}) = S_{xx}(e^{j\omega}) |H(e^{j\omega})|^2$$

$$\text{white input: } S_{xx}(e^{j\omega}) = \sigma^2$$

Interpretation

A lowpass H makes low-frequency noise survive and high-frequency noise shrink. The output is no longer spectrally flat.



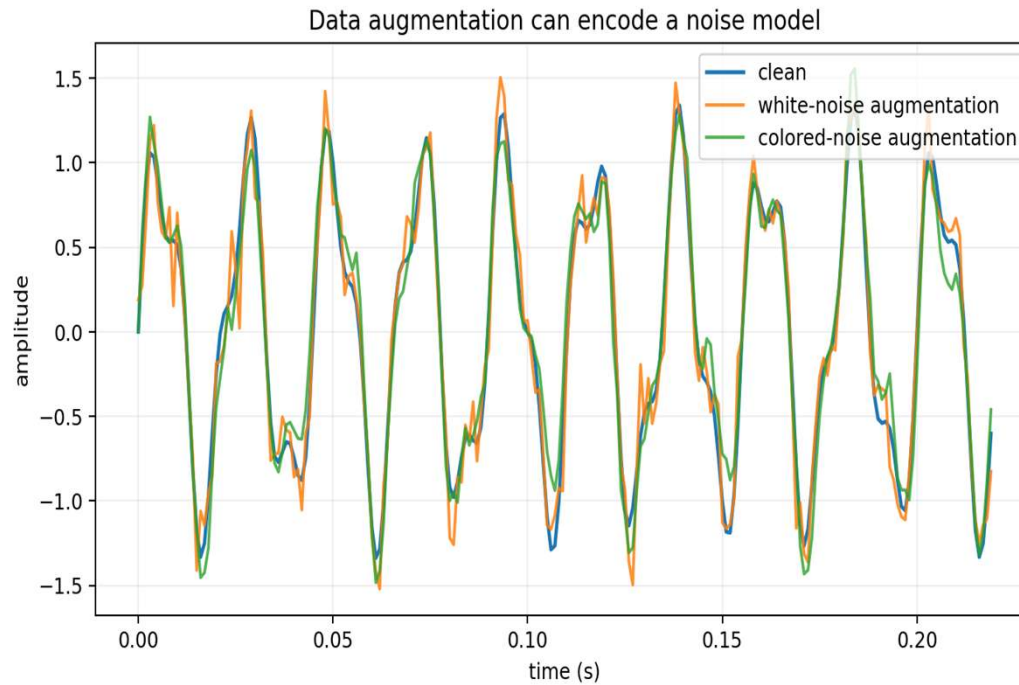
04

ML bridge: noise augmentation

Noise injection is a modeling decision, not just “add random numbers.”

Synthetic noise augmentation in PyTorch

Match the nuisance process you expect the model to encounter.



- White Gaussian noise: simple baseline; independent samples.
- Colored noise: filter white noise to create temporal correlation.
- Scale noise by target SNR or by a random severity range.
- Apply augmentation only to training data, not validation/test data.

ML principle

Augmentation should preserve the label while varying nuisance factors the deployed model will face.

Coding prompt: white vs colored augmentation

Use the same clean waveform and the same noise RMS.

A • White

```
noise = torch.randn_like(x)
```

B • Colored

Generate white noise, then filter it with a stable FIR/IIR.

C • Compare

Plot ACF / PSD and listen or inspect waveforms.

- Keep RMS equal: differences then come from correlation / spectrum, not total power.
- Train a small classifier with each augmentation and compare robustness to matched and mismatched test noise.
- Ask whether the augmentation distribution is physically plausible for the sensing problem.

Lecture 16 wrap-up

An LTI system transforms second-order statistics in a predictable way.

- Mean is multiplied by the filter DC gain $\sum h[k]$.
- Output variance depends on the input covariance and the filter energy / structure.
- With $r_{xy}[m] = E\{X[n]Y[n+m]\}$, $r_{xy} = r_{xx} * h$ for real h .
- Output autocorrelation is $r_{yy} = r_{xx} * h * h_{\text{rev}}$.
- Filtering white noise creates correlated, spectrally colored noise.
- Noise augmentation should encode realistic nuisance statistics.

EXIT QUESTION

White noise with variance 4 drives $h[n] = \{1/2, 1/2\}$. What is the output variance?

Good answer

$\sigma_y^2 = 4[(1/2)^2 + (1/2)^2] = 2$. The lowpass reduces total noise variance while introducing correlation.

References and instructor resources

- Oppenheim & Schafer, Discrete-Time Signal Processing: random signals, correlation, and LTI systems.
- Raschka et al., Machine Learning with PyTorch and Scikit-Learn: preprocessing, tensors, and training workflows.
- Accompanying notebook: `Week8_Random_Signals_DSP_ML_Demos.ipynb`.
- Homework and weekly quiz: `Week8_Random_Signals_Homework_Quiz_Solutions.docx`.